

Universidad San Carlos de Guatemala
Curso: Introducción a la Programación 2

Transportes MRK

Sistema de Gestión de Flota de Buses Extraurbanos

Manual Técnico

Carnet: 202330664

Nombre: Emanuel de Jesus Sosof Peneleu

Fecha: 17/09/26

1. Introducción

Transportes MRK es un sistema web para la gestión de una empresa de transporte interurbano de pasajeros. Con él, la empresa controla su flota de buses, a sus choferes, las rutas y los viajes que se programan cada día. Pero no solo eso: el sistema también es la vitrina donde los clientes buscan un viaje, eligen su asiento, pagan su boleto con una cartera digital y hasta contratan un alquiler privado de bus con chofer.

Este manual técnico explica **cómo está construido** el proyecto por dentro: las tecnologías, la arquitectura, los paquetes y clases, el diseño de la base de datos, las decisiones de diseño y, sobre todo, **los pasos exactos para ponerlo en marcha** en una máquina nueva. Está pensado para que cualquier persona con conocimientos básicos de Java y MySQL pueda compilar el proyecto, cargar su base de datos y ejecutarlo sin perderse.

El sistema maneja cuatro tipos de usuario, cada uno con permisos distintos: **Administrador del Sistema**, **Administrador de Sucursal**, **Chofer** y **Cliente**. Toda la aplicación se construyó siguiendo el patrón MVC y usando Java EE (Jakarta) con Servlets y JSP sobre Maven y Apache Tomcat.

2. Tecnologías utilizadas

El proyecto se construyó con el siguiente conjunto de tecnologías, todas de uso estándar en el desarrollo web con Java:

Tecnología	Versión / Detalle	Para qué se usa
Java	Java 17 (fuente y destino de compilación)	Lenguaje de programación de todo el servidor.
Java EE / Jakarta EE	Jakarta EE API 11.0.0-M1 (scope <code>provided</code>)	Servlets (<code>HttpServlet</code>), sesiones HTTP y JSP.
Servidor web	Apache Tomcat (integrado desde NetBeans)	Ejecuta la aplicación web (.war) y atiende las peticiones.
JSP	Jakarta Server Pages	Vistas: las páginas que ve el usuario y los formularios.
MySQL	MySQL 8.0.46 (Ubuntu)	Base de datos relacional <code>transporte_extraurbano</code> .
JDBC + Conector	<code>mysql-connector-j</code> 9.7.0	Conexión de Java con MySQL desde la clase <code>Conexion</code> .
Maven	Plugins compiler 3.12.1 y war 3.4.0	Gestión de dependencias y empaquetado en <code>.war</code> .
IDE	Apache NetBeans	Edición, ejecución y despliegue del proyecto.
Git / GitHub	Control de versiones	Historial de cambios por caso de uso.

3. Arquitectura del proyecto

El proyecto sigue el patrón **MVC (Modelo – Vista – Controlador)**, separando claramente los datos, la lógica de negocio y la presentación. La dependencia entre capas va en una sola dirección y el ciclo de una petición es el siguiente:

1. El usuario hace clic en un formulario o enlace que apunta a un **Servlet** (patrón `/SvNombre`).
2. El Servlet valida la sesión y el rol, lee los parámetros y delega la lógica en la capa de **Servicio**.
3. El Servicio aplica las reglas de negocio (o lanza `ValidacionException` si algo está mal) y llama al **DAO**.
4. El DAO ejecuta el SQL con `PreparedStatement` y devuelve los modelos (`Usuario` , `Bus` , `Viaje` ...).
5. El Servlet guarda los resultados como atributos de `request` / `session` y reenvía a una **JSP**.

6. La JSP genera el HTML que ve el cliente.

3.1 Flujo de ejemplo: iniciar sesión

El formulario `login.jsp` envía por POST a `SvLogin`. El servlet consulta `UsuarioService.autenticar(correo, password)`, el servicio pregunta a `UsuarioDAO.obtenerPorCorreo()` y compara la contraseña. Si el usuario existe, está activo y la contraseña coincide, se guarda en la sesión HTTP y se lo redirige al panel según su rol (cliente, admin de sucursal, admin del sistema o chofer). En caso contrario se muestra un mensaje de error en el mismo `login.jsp`.

3.2 Paquetes del proyecto

Dentro de `com.mycompany.transportes` el código se organiza en cinco paquetes, cada uno con una responsabilidad puntual:

Paquete	Responsabilidad
<code>conexion</code>	Única clase <code>Conexion.java</code> : abre y cierra las conexiones a MySQL.
<code>modelo</code>	10 clases que representan las entidades del negocio (los registros de la BD).
<code>dao</code>	10 DAOs: cada uno encapsula las consultas SQL de su entidad.
<code>servicio</code>	10 Services: reglas de negocio, validaciones y transacciones.
<code>servlets</code>	12 controladores: reciben la petición, orquestan y reenvían a las vistas.
<code>webapp</code>	Las vistas JSP (páginas, paneles, formularios y publicidad del sitio).

4. Estructura del proyecto

4.1 Capa de modelo (`modelo`) — 10 clases

Clase	Entidad (tabla)	Función
<code>Sucursal</code>	sucursal	Sedes de la empresa (origen de cada viaje).
<code>Usuario</code>	usuario	Todos los usuarios: clientes, choferes y administradores, con su rol y saldo de cartera.
<code>Chofer</code>	chofer	Datos propios del chofer, ligado 1 a 1 con su usuario por el DPI.
<code>Bus</code>	bus	Flota: placa, modelo, capacidad, kilometraje y disponibilidad.
<code>Ruta</code>	ruta	Rutas entre sucursales, con distancia y precio del boleto.
<code>Viaje</code>	viaje	Viajes regulares y de alquiler con sus horarios, kilometrajes y estado operativo.
<code>Boleto</code>	boleto	Boletos vendidos: asiento, precio pagado y cliente.
<code>AlquilerPrivado</code>	alquiler_privado	Detalle del alquiler: origen, destino, fechas, pasajeros y precio.
<code>Mantenimiento</code>	mantenimiento_taller	Gastos de taller y repuestos por bus.
<code>ViajeDisponible</code>	vista de viaje+bus	Proyección para la consulta pública de viajes (ruta, hora, precio, asientos libres).

4.2 Capa de acceso a datos (`dao`) — 10 DAOs

DAO	Función
<code>Conexion</code> (<code>conexion</code>)	Da la conexión con la base de datos (URL, usuario y clave), leyendo <code>db.properties</code> .
<code>SucursalDAO</code>	Crear, listar, obtener por id, actualizar, desactivar y activar sucursales.
<code>UsuarioDAO</code>	Crear usuarios, buscar por DPI o correo, listar por rol o sucursal, actualizar, desactivar/activar, recargar saldo y actualizar perfil.
<code>ChoferDAO</code>	CRUD de choferes; consultas con JOIN a <code>usuario</code> para traer nombre, teléfono, dirección y estado.
<code>BusDAO</code>	CRUD de buses y consulta de buses con viajes activos (para desactivación segura).
<code>RutaDAO</code>	CRUD de rutas; listar con los nombres de origen/destino (JOIN a sucursal).
<code>ViajeDAO</code>	CRUD de viajes, asignar bus/chofer, registrar salida y llegada (con validaciones), eliminar.
<code>BoletoDAO</code>	Vender boleto con control de asientos ocupados y consultas por viaje.
<code>AlquilerPrivadoDAO</code>	Guardar solicitud de alquiler, confirmar/cambiar precio, asignar bus y chofer.
<code>MantenimientoTallerDAO</code>	Registrar y consultar gastos de taller y repuestos.
<code>ConfiguracionSistemaDAO</code>	Leer y actualizar el monto de depreciación por kilómetro.

4.3 Capa de servicios (servicio) — 10 Services

Service	Función
UsuarioService	Autenticación, registro de cliente/chofer/admin de sucursal (con transacción), desactivación con la regla del último admin de sistema, perfil.
BusService	CRUD de buses y regla de desactivación si tiene viajes programados o en tránsito.
ChoferService	CRUD de choferes, disponibilidad y consultas detalladas con usuario.
RutaService	CRUD de rutas y la regla de no eliminar rutas con viajes asociados.
ViajeService	CRUD de viajes, asignación de bus/chofer, salidas y llegadas.
ViajeRegularService	Consulta pública de viajes regulares disponibles con sus asientos libres.
ComprarBoletoService	Compra de boletos con pago desde la cartera y control de capacidad.
AlquilerService	Solicitud, confirmación de precio, asignación y pago del alquiler privado.
MantenimientoService	Registro y listado de gastos de taller.
SucursalService	CRUD de sucursales (usado por el admin del sistema).

4.4 Capa de controladores (`servlets`) — 12 Servlets

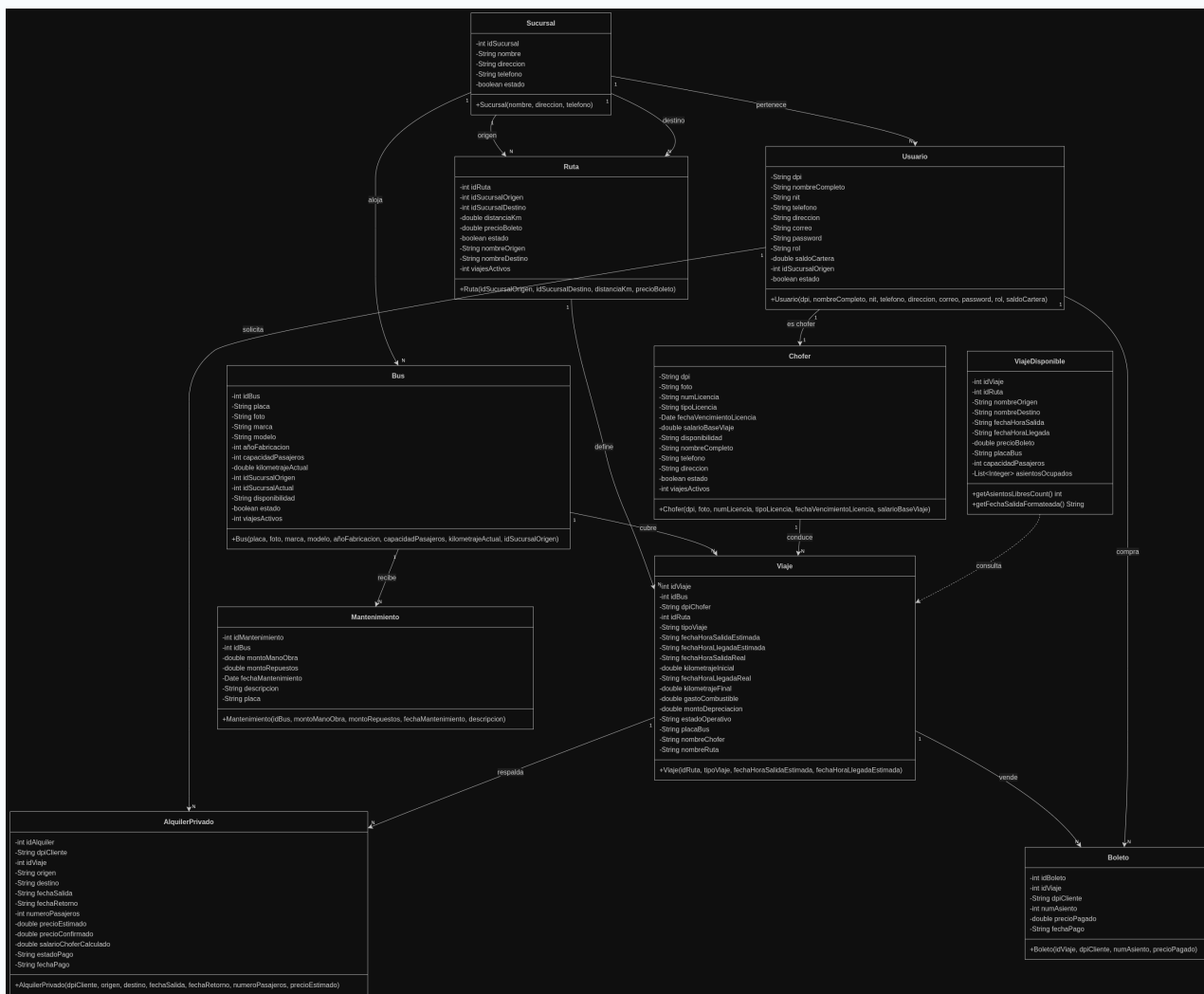
Servlet	URL	Función
<code>SvLogin</code>	<code>/SvLogin</code>	Autenticar usuarios, cerrar sesión y redirigir según el rol.
<code>SvUsuario</code>	<code>/SvUsuario</code>	Registro de clientes, choferes, admin de sucursal, recarga de cartera y edición de perfil.
<code>SvBus</code>	<code>/SvBus</code>	CRUD de buses.
<code>SvChofer</code>	<code>/SvChofer</code>	CRUD de choferes (listar, editar, desactivar, activar).
<code>SvRuta</code>	<code>/SvRuta</code>	CRUD de rutas.
<code>SvViaje</code>	<code>/SvViaje</code>	CRUD de viajes, registro de salida y llegada.
<code>SvViajeRegular</code>	<code>/SvViajeRegular</code>	Listar viajes disponibles y gestionar la selección de asientos.
<code>SvBoleto</code>	<code>/SvBoleto</code>	Compra y pago de boletos con cartera.
<code>SvAlquiler</code>	<code>/SvAlquiler</code>	Solicitud, confirmación, asignación y pago de alquileres privados.
<code>SvMantenimiento</code>	<code>/SvMantenimiento</code>	Registro y listado de gastos de taller.
<code>SvSucursal</code>	<code>/SvSucursal</code>	CRUD de sucursales (admin del sistema).
<code>SvAdminSistema</code>	<code>/SvAdminSistema</code>	CRUD de admins de sucursal, configuración de depreciación y gestión de usuarios.

4.5 Vistas (webapp) — JSPs principales

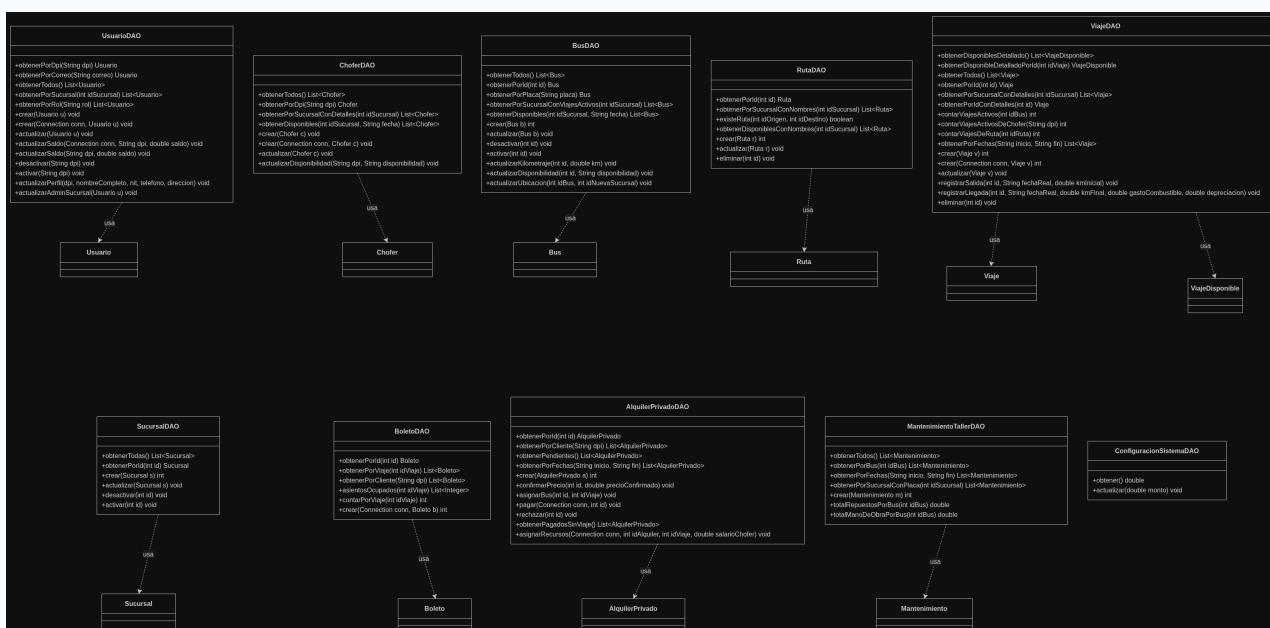
Área	Vistas JSP
Públicas	<code>index.jsp</code> (consultas de viajes), <code>login.jsp</code> , <code>registro.jsp</code> , <code>quienesSomos.jsp</code> , <code>servicios.jsp</code> , <code>header.jsp</code> (menú y sesión).
Cliente	<code>home.jsp</code> , <code>viajesRegulares.jsp</code> , <code>seleccionAsientos.jsp</code> , <code>pagarBoleto.jsp</code> , <code>compraExitosa.jsp</code> , <code>recargar.jsp</code> , <code>miPerfil.jsp</code> , <code>alquilerPrivado.jsp</code> , <code>misAlquileres.jsp</code> , <code>alquilerSolicitudExito.jsp</code> , <code>pagoAlquilerExito.jsp</code> .
Admin de Sucursal	<code>panelAdminSucursal.jsp</code> , <code>gestionarBuses.jsp</code> , <code>formularioBus.jsp</code> , <code>gestionarChoferes.jsp</code> , <code>formularioChofer.jsp</code> , <code>crearChofer.jsp</code> , <code>gestionarRutas.jsp</code> , <code>formularioRuta.jsp</code> , <code>gestionarViajes.jsp</code> , <code>formularioViaje.jsp</code> , <code>registrarSalida.jsp</code> , <code>registrarLlegada.jsp</code> , <code>mantenimientos.jsp</code> , <code>registrarMantenimiento.jsp</code> , <code>alquileresPendientes.jsp</code> , <code>asignarAlquiler.jsp</code> , <code>alquileresPagados.jsp</code> .
Admin de Sistema	<code>panelAdminSistema.jsp</code> , <code>gestionarSucursales.jsp</code> , <code>formularioSucursal.jsp</code> , <code>gestionarAdminSucursal.jsp</code> , <code>formularioAdminSucursal.jsp</code> , <code>configuracionSistema.jsp</code> , <code>gestionarUsuarios.jsp</code> .
Chofer	<code>panelChofer.jsp</code> .

4.6 Diagramas de clases por capa

Los siguientes diagramas de clases muestran los atributos y los métodos principales de cada capa. Se omiten los métodos accesorios (getters y setters) para que la lectura no se sature y solo se aprecie la lógica de cada clase.



Capa de modelo: las 10 entidades y sus relaciones.



Capa de acceso a datos: los 10 DAOs con sus métodos principales.



Capa de servicios: los 10 Services y su relación con los DAOs.

5. Base de datos

El sistema usa una base de datos relacional llamada `transporte_extraurbano` con 10 tablas. El script de creación es `transporte_extraurbano.sql` y los datos de prueba para probar todo de inmediato están en `datos_seed.sql`.

5.1 Diagrama Entidad-Relación

La siguiente imagen muestra las 10 tablas de la base de datos junto con sus llaves primarias y foráneas:

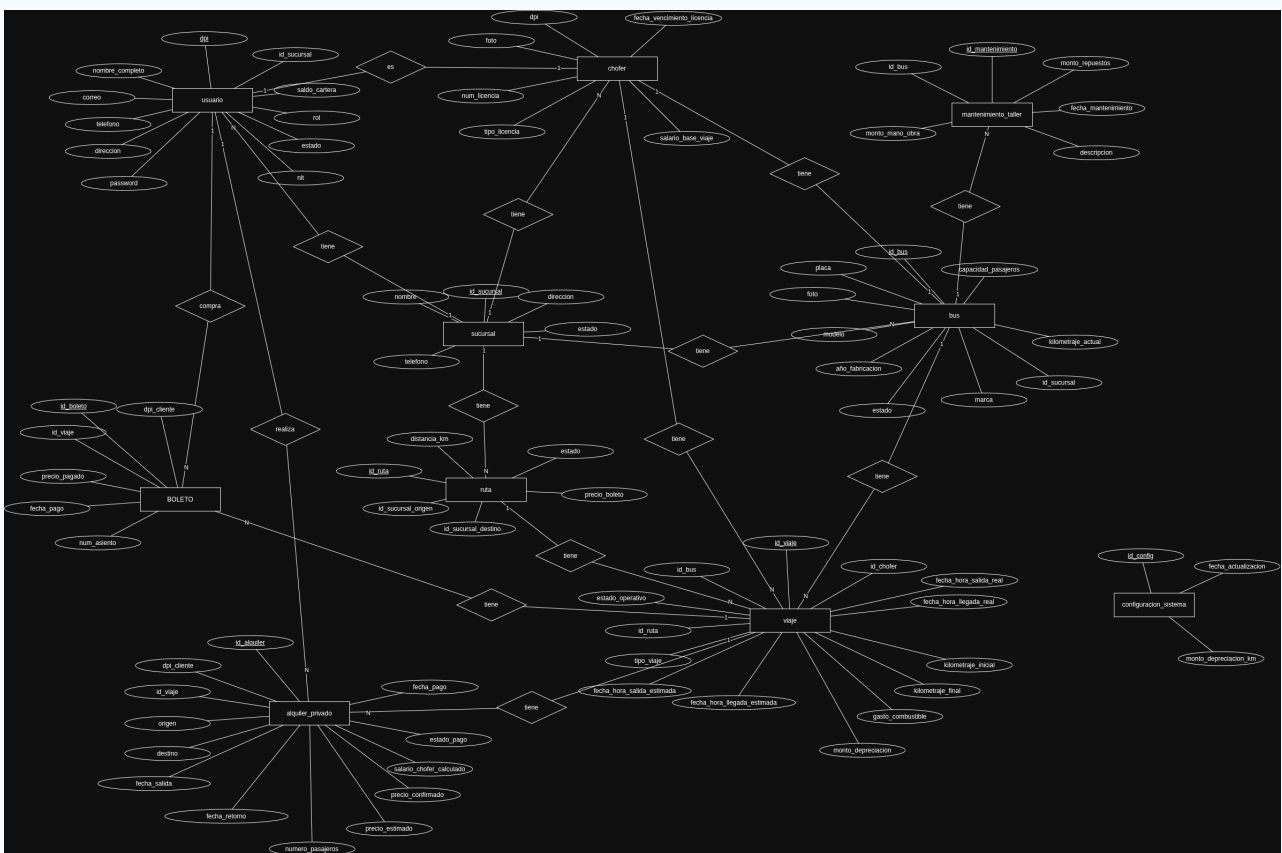


Diagrama Entidad-Relación de la base de datos `transporte_extraurbano`.

5.2 Tablas en MySQL

CONFIGURACION_SISTEMA

Campo	Tipo	Llave	Descripción
id_config	INT AUTO_INCREMENT	PK	Identificador del parámetro.
monto_depreciacion_km	DECIMAL(10,2)		Monto de depreciación por km (valor inicial 1.50).
fecha_actualizacion	DATETIME		Cuándo se actualizó por última vez.

SUCURSAL

Campo	Tipo	Llave	Descripción
id_sucursal	INT AUTO_INCREMENT	PK	Identificador de la sede.
nombre	VARCHAR(80)	UNIQUE	Nombre de la sede.
direccion	VARCHAR(150)		Dirección.
telefono	VARCHAR(15)		Teléfono (formato libre, ej. 2233-4455).
estado	BOOLEAN		1 activa, 0 desactivada.

USUARIO

Campo	Tipo	Llave	Descripción
dpi	VARCHAR(13)	PK	Documento de identidad (13 dígitos).
nombre_completo	VARCHAR(100)		Nombre completo.
nit	VARCHAR(15)		NIT del usuario (8 dígitos al registrarse).
telefono	VARCHAR(15)		Teléfono (8 dígitos al registrarse).
direccion	VARCHAR(150)		Dirección.
correo	VARCHAR(80)	UNIQUE	Correo usado para iniciar sesión.
password	VARCHAR(255)		Contraseña.
rol	ENUM(ADMIN_SISTEMA, ADMIN_SUCURSAL, CLIENTE, CHOFER)		Papel del usuario en el sistema.
saldo_cartera	DECIMAL(10,2)		Saldo de la cartera digital (≥ 0 por CHECK).
id_sucursal_origen	INT NULL	FK → sucursal	Sucursal a la que pertenece (nula para admin de sistema).
estado	BOOLEAN		1 activo, 0 desactivado.

BUS

Campo	Tipo	Llave	Descripción
id_bus	INT AUTO_INCREMENT	PK	Identificador.
placa	VARCHAR(10)	UNIQUE	Placa del bus.
foto	VARCHAR(255) NULL		URL de la foto del bus.
marca	VARCHAR(50)		Marca.
modelo	VARCHAR(50)		Modelo.
año_fabricacion	INT		Año de fabricación.
capacidad_pasajeros	INT		Cuántos pasajeros caben.
kilometraje_actual	DECIMAL(12,2)		Kilómetros recorridos a la fecha.
id_sucursal_origen	INT	FK → sucursal	Sucursal dueña del bus.
id_sucursal_actual	INT	FK → sucursal	Sucursal donde se encuentra ahora.
disponibilidad	ENUM(Disponible, Ocupado, Taller)		Estado operativo.
estado	BOOLEAN		1 activo, 0 desactivado.

CHOFER

Campo	Tipo	Llave	Descripción
dpi	VARCHAR(13)	PK + FK → usuario(dpi)	DPI del chofer (1 a 1 con su usuario).
foto	VARCHAR(255) NULL		URL de la foto.
num_licencia	VARCHAR(20)		Número de licencia.
tipo_licencia	ENUM(A, B)		Tipo de licencia.
fecha_vencimiento_licencia	DATE		Vencimiento de la licencia.
salario_base_viaje	DECIMAL(10,2)		Salario base por viaje regular.
disponibilidad	ENUM(Disponible, Ocupado)		Si tiene o no viaje activo.

RUTA

Campo	Tipo	Llave	Descripción
id_ruta	INT AUTO_INCREMENT	PK	Identificador.
id_sucursal_origen	INT	FK → sucursal	Sucursal origen.
id_sucursal_destino	INT	FK → sucursal	Sucursal destino.
distancia_km	DECIMAL(10,2)		Distancia en kilómetros.
precio_boleto	DECIMAL(10,2)		Precio del boleto.
estado	BOOLEAN		1 activa, 0 desactivada.

VIAJE

Campo	Tipo	Llave	Descripción
id_viaje	INT AUTO_INCREMENT	PK	Identificador.
id_bus	INT	FK → bus	Bus asignado (se asigna al crear o después).
dpi_chofer	VARCHAR(13)	FK → chofer	Chofer asignado.
id_ruta	INT NULL	FK → ruta	Ruta (nula en alquileres, se usa origen/destino del alquiler).
tipo_viaje	ENUM(REGULAR, ALQUILER_PRIVADO)		Tipo del viaje (no se puede cambiar).
fecha_hora_salida_estimada	DATETIME		Salida estimada.
fecha_hora_llegada_estimada	DATETIME		Llegada estimada.
fecha_hora_salida_real	DATETIME NULL		Salida real registrada.
kilometraje_inicial	DECIMAL(12,2) NULL		Km al salir.
fecha_hora_llegada_real	DATETIME NULL		Llegada real registrada.
kilometraje_final	DECIMAL(12,2) NULL		Km al llegar.
gasto_combustible	DECIMAL(10,2) NULL		Combustible gastado.
monto_depreciacion	DECIMAL(10,2) NULL		Depreciación calculada por km.
estado_operativo	ENUM(PROGRAMADO, EN_TRANSITO, FINALIZADO, CANCELADO)		Estado del viaje.

BOLETO

Campo	Tipo	Llave	Descripción
id_boleto	INT AUTO_INCREMENT	PK	Identificador.
id_viaje	INT	FK → viaje	Viaje del boleto.
dpi_cliente	VARCHAR(13)	FK → usuario	Cliente comprador.
num_asiento	INT	UNIQUE(id_viaje, num_asiento)	Número de asiento (sin repetir por viaje).
precio_pagado	DECIMAL(10,2)		Precio que pagó.
fecha_pago	DATETIME		Cuándo pagó.

ALQUILER_PRIVADO

Campo	Tipo	Llave	Descripción
id_alquiler	INT AUTO_INCREMENT	PK	Identificador.
dpi_cliente	VARCHAR(13)	FK → usuario	Cliente que solicita.
id_viaje	INT NULL	FK → viaje	Viaje asociado (si ya se creó).
origen / destino	VARCHAR(100)		Puntos del recorrido.
fecha_salida	DATETIME		Cuándo sale.
fecha_retorno	DATETIME NULL		Cuándo vuelve (opcional).
numero_pasajeros	INT		Cuántas personas viajan.
precio_estimado	DECIMAL(10,2)		Precio calculado al solicitar.
precio_confirmado	DECIMAL(10,2) NULL		Precio confirmado/cambiado por el admin.
salario_chofer_calculado	DECIMAL(10,2) NULL		Salario del chofer con 15% adicional.
estado_pago	ENUM(SOLICITADO, CONFIRMADO, PAGADO, RECHAZADO)		Etapas del alquiler.
fecha_pago	DATETIME NULL		Cuándo se pagó.

MANTENIMIENTO_TALLER

Campo	Tipo	Llave	Descripción
id_mantenimiento	INT AUTO_INCREMENT	PK	Identificador.
id_bus	INT	FK → bus	Bus al que se hizo el mantenimiento.
monto_mano_obra	DECIMAL(10,2)		Mano de obra.
monto_repuestos	DECIMAL(10,2)		Repuestos.
fecha_mantenimiento	DATE		Fecha del mantenimiento.
descripcion	TEXT		Detalle/descripción del trabajo.

5.3 Valores de tipo (ENUM) usados en la base

Tabla.Columna	Valores
usuario.rol	ADMIN_SISTEMA , ADMIN_SUCURSAL , CLIENTE , CHOFER
bus.disponibilidad	Disponible , Ocupado , Taller
chofer.disponibilidad	Disponible , Ocupado
viaje.tipo_viaje	REGULAR , ALQUILER_PRIVADO
viaje.estado_operativo	PROGRAMADO , EN_TRANSITO , FINALIZADO , CANCELADO
alquiler_privado.estado_pago	SOLICITADO , CONFIRMADO , PAGADO , RECHAZADO

5.4 Relaciones principales

Relación	De	A	Cardinalidad
Pertenece	USUARIO	SUCURSAL (id_sucursal_origen)	N – 1
Es chofer	CHOFER	USUARIO (dpi)	1 – 1
Se asigna	VIAJE	BUS / CHOFER	N – 1
Tiene ruta	VIAJE	RUTA	N – 1
Vende	BOLETO	VIAJE / USUARIO	N – 1
Solicita	ALQUILER_PRIVADO	USUARIO / VIAJE	N – 1
Recibe	MANTENIMIENTO_TALLER	BUS	N – 1
Configura	CONFIGURACION_SISTEMA	—	1 fila global

6. Decisiones de diseño

A continuación se explican las decisiones de diseño más importantes del sistema y el motivo de cada una:

6.1 El chofer es primero un usuario

En vez de duplicar nombre, teléfono y dirección en la tabla `chofer`, el chofer **comparte el DPI** con su fila en `usuario` (relación 1 a 1). La tabla `chofer` solo guarda lo propio del puesto: licencia, salario y disponibilidad. Esto evita datos repetidos y hace que el chofer también pueda actuar como cliente en el sistema.

6.2 El viaje se puede crear sin bus ni chofer

Al registrar un viaje nuevo no siempre se sabe de inmediato qué bus y chofer lo cubrirán. Por eso las columnas `id_bus` y `dpi_chofer` permiten valores nulos y se asignan más adelante (en la edición del viaje o al asignar un alquiler) con un `UPDATE`. La salida del viaje exige que ambos ya estén asignados.

6.3 Salario del chofer con 15% en alquiler privado

En un viaje regular el chofer gana su `salario_base_viaje`. En un alquiler privado gana un 15% adicional, es decir `salario_base_viaje × 1.15`. El monto se guarda en `alquiler_privado.salario_chofer_calculado` cuando el administrador asigna el chofer.

6.4 Nada se borra de verdad: todo se desactiva

Los usuarios, choferes, buses, rutas y sucursales tienen la columna `estado`. En lugar de eliminar un registro, el sistema lo **desactiva**; así se conserva su historial (boletos, viajes, cuentas). Además, la desactivación está regulada: un bus solo se desactiva si no tiene viajes programados/en tránsito, una ruta solo si no tiene viajes, y a nivel de sistema existe la regla de que **siempre debe quedar al menos un administrador de sistema activo**.

6.5 Integridad referencial con llaves foráneas

Todas las tablas usan llaves foráneas con `ON DELETE RESTRICT`: no se puede borrar un registro al que otro hace referencia. Las fechas y montos se controlan con `CHECK` (por ejemplo, el saldo de la cartera nunca es negativo y el asiento de un boleto siempre es ≥ 1).

7. Cómo se conecta la base de datos

Todas las conexiones pasan por la clase estática `Conexion`. Primero se intenta leer un archivo de configuración `db.properties`; si no existe, se usan la URL, usuario y clave por defecto. Luego carga el driver de MySQL y devuelve una conexión lista para los DAO.

```
public static Connection obtener() throws SQLException {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
    } catch (ClassNotFoundException e) {
        throw new SQLException("No se pudo cargar el driver de MySQL: " + e.getMessage());
    }
    return DriverManager.getConnection(url, usuario, password);
}
```

Valores por defecto (sobrescribibles con `db.properties`):

Parámetro	Valor
URL	<code>jdbc:mysql://localhost:3306/transporte_extraurbano?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true</code>
Usuario	<code>mrk</code>
Contraseña	<code>scp049</code>

7.1 Archivo `db.properties` (opcional)

Si se prefiere cambiar la configuración sin tocar código, se crea un archivo `db.properties` junto a la carpeta del proyecto:

```
url=jdbc:mysql://localhost:3306/transporte_extraurbano?useSSL=false&serverTimezone=UTC
usuario=mrk
password=scp049
```

8. Consultas principales (SQL)

Todas las consultas se ejecutan con `PreparedStatement` para evitar inyección SQL. Algunos ejemplos representativos de lo que hace el sistema:

8.1 Buscar un usuario por correo (login)

```
SELECT * FROM usuario WHERE correo = ?;
```

8.2 Listar choferes de una sucursal con sus datos de usuario (nombre, teléfono, dirección, estado)

```
SELECT c.dpi, c.num_licencia, c.tipo_licencia, c.fecha_vencimiento_licencia,  
       c.salario_base_viaje, u.nombre_completo, u.telefono, u.direccion, u.estado  
FROM chofer c  
JOIN usuario u ON u.dpi = c.dpi  
WHERE u.id_sucursal_origen = ?;
```

8.3 Choferes disponibles (sin viaje activo y con usuario activo)

```
SELECT c.dpi, u.nombre_completo  
FROM chofer c  
JOIN usuario u ON u.dpi = c.dpi  
WHERE u.estado = TRUE AND c.disponibilidad = 'Disponible'  
      AND c.dpi NOT IN (  
        SELECT dpi_chofer FROM viaje  
        WHERE estado_operativo IN ('PROGRAMADO', 'EN_TRANSITO')  
      );
```

8.4 Vender un boleto y descontar la cartera (transacción)

```
-- 1) Verificar que el asiento esté libre en el viaje  
SELECT COUNT(*) AS ocupados FROM boleto WHERE id_viaje = ? AND num_asiento = ?;  
  
-- 2) Descontar el saldo  
UPDATE usuario SET saldo_cartera = saldo_cartera - ? WHERE dpi = ? AND saldo_cartera > 0;  
  
-- 3) Insertar el boleto  
INSERT INTO boleto (id_viaje, dpi_cliente, num_asiento, precio_pagado) VALUES (?, ?, ?, ?);
```

8.5 Registrar llegada de un viaje con su depreciación

```
UPDATE viaje
SET fecha_hora_llegada_real = ?, kilometraje_final = ?,
    gasto_combustible = ?, monto_depreciacion = ROUND((? - kilometraje_inicial) * ?,
    estado_operativo = 'FINALIZADO'
WHERE id_viaje = ?;
```

(el último parámetro, el monto de depreciación por km, viene de `configuracion_sistema.monto_depreciacion_km`).

8.6 Alquiler privado: salario del chofer con 15%

```
SELECT ROUND(c.salario_base_viaje * 1.15, 2) AS salario_chofer
FROM chofer c WHERE c.dpi = ?;
```

9. Seguridad y validaciones

- **Control de acceso por rol:** cada servlet verifica la sesión y el rol antes de ejecutar cualquier acción (por ejemplo, `SvSucursal` y `SvAdminSistema` exigen rol `ADMIN_SISTEMA`; `SvBus`, `SvChofer`, etc. exigen `ADMIN_SUCURSAL`). Si no hay sesión válida se redirige a `login.jsp`.
- **Validación en el backend:** la capa de Servicios valida con `ValidacionException` todos los campos: DPI de 13 dígitos, NIT y teléfono de exactamente 8 dígitos, contraseña de al menos 6 caracteres, correo único, fechas futuras para vencimientos, salario mayor a 0, montos de depreciación mayores a 0, etc.
- **Contraseñas por el momento en texto plano** en la base (se comparan directamente en el login), alineado con la simpleza del alcance del proyecto.
- **SQL seguro:** 100% `PreparedStatement` con parámetros `?`, sin concatenar texto del usuario.
- **Usuarios no se eliminan:** solo se desactivan (`estado = FALSE`) y un usuario inactivo no puede iniciar sesión.
- **Regla crítica:** el sistema impide desactivar al último administrador del sistema activo.
- **Transacciones:** las operaciones que tocan varias tablas (registrar chofer, pagar boleto, asignar alquiler) usan `Connection` con `setAutoCommit(false)` y `rollback()` si algo falla.

10. Requisitos e instalación

10.1 Requisitos del equipo

- **Java 17** (o superior) instalado y en el `PATH`.
- **NetBeans** con soporte para Java EE / Tomcat (o Apache Tomcat 10+ configurado).
- **MySQL 8** corriendo en el equipo.
- **Maven** (NetBeans lo trae integrado; también se puede usar desde consola).
- Conexión a internet la primera vez, para que Maven descargue las dependencias (`mysql-connector-j`).

10.2 Preparar la base de datos

1. Crear la base y sus tablas con el script de estructura. En la terminal de MySQL:

```
mysql -u tu_usuario -p < transporte_extraurbano.sql
```

2. Cargar los datos de ejemplo (1 admin de sistema, 3 admins de sucursal, 9 choferes, clientes, buses, rutas y viajes para probar todo de inmediato):

```
mysql -u tu_usuario -p transporte_extraurbano < datos_seed.sql
```

3. Verificar la conexión. Si tu usuario y contraseña de MySQL no son `mrk` / `scp049`, crea un archivo `db.properties` junto al proyecto con tu `url`, `usuario` y `password` (ver sección 7.1) para que el sistema use la tuya.

10.3 Ejecutar el programa desde NetBeans

1. Abrir NetBeans → **File** → **Open Project** y seleccionar la carpeta `transportes`.
2. Esperar a que Maven resuelva las dependencias (la primera vez descarga `jakarta.jakartaee-api` y `mysql-connector-j`).
3. Confirmar que el servidor Tomcat esté agregado en **Services** → **Servers**.
4. Click derecho sobre el proyecto → **Run** (o presionar **F6**).
5. NetBeans compila, empaqueta el `.war` y lo despliega en Tomcat.
6. Abrir el navegador en la URL del proyecto, típicamente:

```
http://localhost:8080/transportes-1.0-SNAPSHOT/
```

(el contexto puede variar según cómo esté configurado Tomcat en tu NetBeans; el nombre del .war es `transportes-1.0-SNAPSHOT` según el `pom.xml`).

7. La vista de inicio (`index.jsp`) muestra los viajes disponibles sin necesidad de estar logueado.

10.4 Ejecutar desde consola con Maven (opcional)

```
mvn clean package
mvn org.apache.tomcat.maven:tomcat7-maven-plugin:2.2:run
```

o bien copiar el archivo `target/transportes-1.0-SNAPSHOT.war` dentro de la carpeta `webapps/` de un Tomcat instalado y reiniciarlo.